Software Architecture and Design Specification

Demand Forecasting & Inventory Planning System

Project	**Demand Forecasting & Inventory Planning System**
Version	1.1
Authors	Team 03 - Rahul Saran Sharma, Vishnu P, Sahana K Golabanvi, K Preksha
Date	12-10-2026
Status	Draft

Revision History

Version	**Date**	**Author**	**Change Summary**
1.0	07-10-2026	Team 03	Initial SAD tailored to assigned mini-project
1.1	12-10-2026	Team 03	Consistency fixes; corrected security mappings; added F-023, forecasting/planning decisions, CSV/API details and updated traceability.

Approvals

Role	**Name**	**Signature / Date**
Course Instructor		
Team Lead		

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Demand Forecasting & Inventory Planning System. It translates the approved SRS requirements into components, interfaces, data flows, security controls, and implementation decisions for Team 03.

1.2 Scope

The design covers user authentication, product and inventory management, historical sales management, demand forecasting, reorder recommendation, stockout/overstock risk identification, dashboards, reports, audit logging, and the supporting database/API structure. Direct supplier purchase-order placement, payment processing, and warehouse hardware control remain outside the initial mini-project scope.

1.3 Audience
Developers, testers, course instructors/evaluators, inventory planners, administrators, and future maintainers of the system.

1.4 Definitions
- DFIP - Demand Forecasting & Inventory Planning.
- SKU - Stock Keeping Unit.
- RBAC - Role-Based Access Control.
- API - Application Programming Interface.
- Forecast Horizon - Future period for which demand is predicted.
- Reorder Quantity - Suggested quantity to replenish based on forecast demand, available stock, and configured planning parameters.

2. Document Overview

2.1 How to Use This Document
Section 3 describes the architecture, major components, technology choices, risks, security, and traceability. Section 4 describes detailed runtime interactions, APIs, error handling, monitoring, and user-experience decisions. The appendices provide glossary, references, and tools.

2.2 Related Documents
- Software Requirements Specification (SRS) - Demand Forecasting & Inventory Planning System v1.1.
- Software Test Plan (STP) - Demand Forecasting & Inventory Planning System v1.1.
- Requirements Traceability Matrix (RTM) contained in the SRS.

3. Architecture

3.1 Goals & Constraints
Architecture goals are correctness, maintainability, modularity, security, acceptable response time, traceability, and easy academic-scale deployment. The design must support the SRS performance targets, preserve committed data, and separate forecasting logic from user-interface and persistence concerns.

- Forecast quality depends on the quality and quantity of historical sales data; at least 8 aggregated periods are required before forecasting.
- The first version is intended for moderate academic-scale data volumes and a small number of concurrent users. Supported forecast horizons are 1-12 weeks or 1-6 months.
- The system should remain deployable on standard Windows/Linux hardware without specialized warehouse devices.
- Sensitive functions must use authenticated, server-side role checks.
- The solution must preserve requirement-to-component and requirement-to-test traceability.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns	Architectural Response
Inventory Planner / Manager	Accurate forecasts, clear reorder decisions, quick access to risk information.	Forecasting engine, planning service, dashboard and alerts.
Administrator	Controlled data changes, user	RBAC, administration functions, audit

Stakeholder	Primary Concerns	Architectural Response
	management, configuration, auditability.	logging.
Developers	Modularity, testability, simple interfaces.	Layered architecture with REST-style APIs and separated services.
Testers / Evaluators	Traceable behavior and measurable acceptance criteria.	Requirement IDs mapped to components, APIs and test cases.

3.3 Component (UML) Diagram

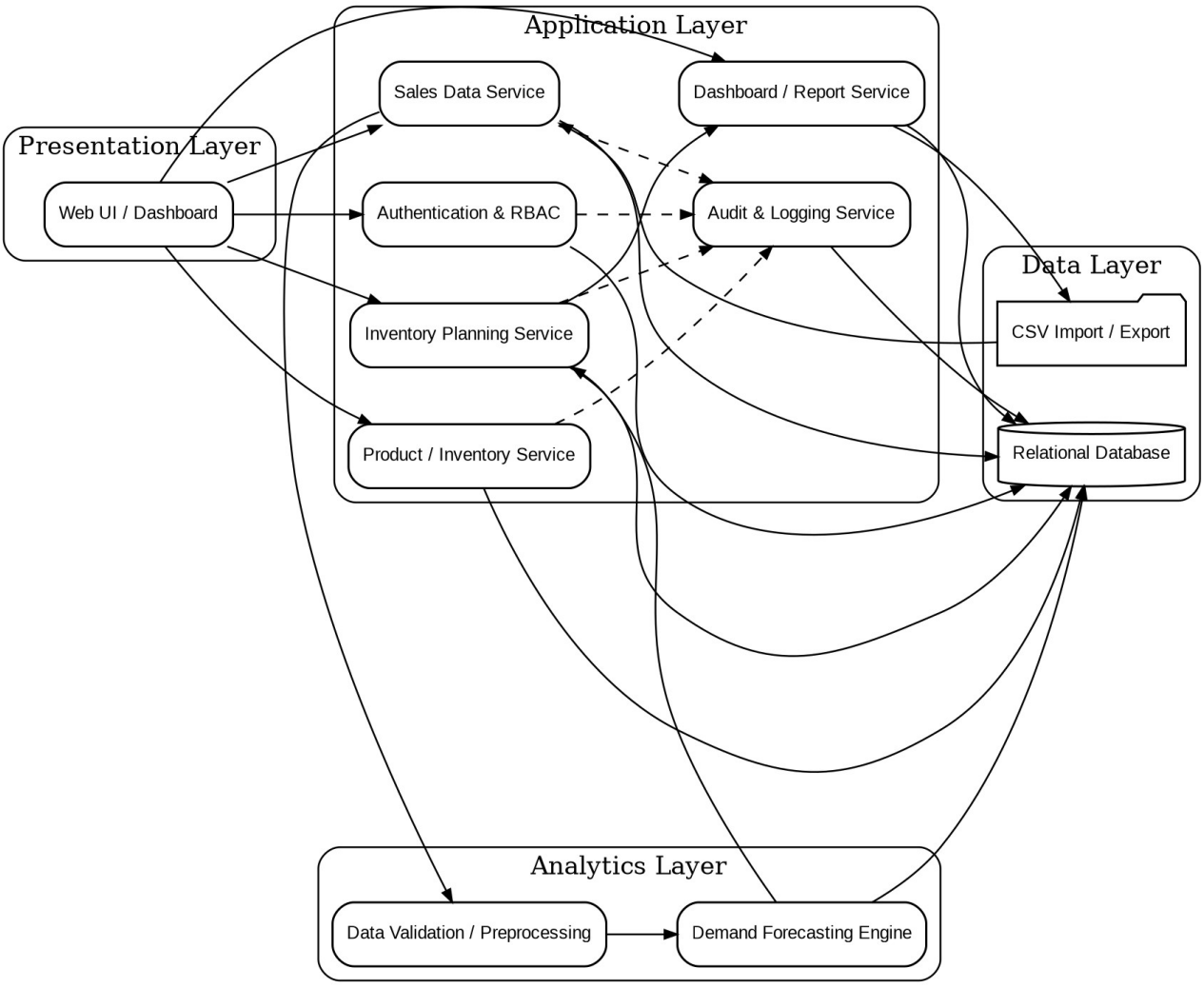


Figure 1. Component view of the Demand Forecasting & Inventory Planning System.

3.4 Component Descriptions

Component	Responsibility	Key Requirement IDs
Web UI / Dashboard	Provides login, data-entry forms, forecast views, inventory-planning views, alerts and reports.	DFIP-F-001, F-003, F-009, F-011, F-017 to F-020
Authentication & RBAC	Authenticates users, maintains sessions and	DFIP-F-001 to F-003, F-021; DFIP-SR-001 to

Component	Responsibility	Key Requirement IDs
	enforces role permissions.	SR-003
Product / Inventory Service	Manages product/SKU records, stock quantities and reorder parameters.	DFIP-F-004 to F-006
Sales Data Service	Captures, validates, imports and queries historical sales data.	DFIP-F-007 to F-009
Data Validation / Preprocessing	Checks required fields, invalid quantities, missing/duplicate values and prepares time-series input.	DFIP-F-006 to F-010; DFIP-SR-004
Demand Forecasting Engine	Preprocesses validated demand history and generates forecasts with model/version metadata; uses Holt-Winters when >= 2 seasonal cycles are available, otherwise simple exponential smoothing.	DFIP-F-010 to F-012; DFIP-NF-002
Inventory Planning Service	Calculates reorder recommendations and classifies stockout/overstock risk using forecast demand, current stock, safety stock and overstock factor.	DFIP-F-013 to F-016
Dashboard / Report Service	Aggregates KPIs, alerts and planning reports; supports export.	DFIP-F-017 to F-020
Audit & Logging Service	Records security-sensitive and data-changing operations.	DFIP-F-022
Administration / Settings Service	Manages users/roles and administrator-configurable planning parameters such as default safety stock, overstock factor and minimum history.	DFIP-F-021, DFIP-F-023; DFIP-SR-003
Deployment / Transport Security	Provides HTTPS/TLS for browser-to-server and API traffic when deployed over a network.	DFIP-SR-005

3.5 Chosen Architecture Pattern and Rationale

A layered modular architecture is chosen. The presentation layer handles user interaction, the application layer implements business operations, the analytics layer performs validation and forecasting, and the data layer manages persistent storage and file exchange. This pattern is suitable for a student mini-project because it provides clear separation of concerns, is easier to test and maintain than a distributed microservices architecture, and still allows the forecasting engine to evolve independently from the web interface.

Microservices were not selected for the first version because independent deployment, service discovery, inter-service security, and distributed monitoring would add complexity without a clear benefit for the expected project scale.

3.6 Technology Stack & Data Stores

Layer / Concern	Proposed Technology	Purpose
Frontend	React or standard HTML/CSS/JavaScript	Responsive web forms, dashboard and charts.

Layer / Concern	Proposed Technology	Purpose
Backend API	Python FastAPI	REST-style endpoints, validation, authentication and business orchestration.
Forecasting / Analytics	Python, Pandas, NumPy, statsmodels	Preprocessing and forecasting: Holt-Winters exponential smoothing when >= 2 seasonal cycles are available; simple exponential smoothing otherwise; negative predictions clipped to 0.
Database	MySQL or PostgreSQL	Users, products, sales, inventory, forecasts, recommendations and audit records.
Data Exchange	CSV and JSON	CSV sales import uses columns sku,date,quantity_sold with date format YYYY-MM-DD; JSON is used for API payloads.
Security	HTTPS/TLS, password hashing, RBAC	Protect credentials, traffic and restricted operations.
Version Control	Git and GitHub	One shared repository for Team 03 source code and project documents.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Insufficient or poor-quality sales history	Forecasts may be misleading.	Validate data, report missing/invalid rows, require minimum history, and display limitations.
Unexpected forecast/model error	Incorrect planning recommendation.	Use deterministic validation, version model settings, and log forecast metadata.
Stale inventory values	Reorder recommendation may be inaccurate.	Show last-updated time and require the latest stock value before planning.
Unauthorized data modification	Loss of integrity or confidentiality.	Authentication, server-side RBAC, password hashing and audit records.
Database failure or data loss	Application unavailable or records lost.	Transaction-safe persistence, regular backup during deployment, and restart/recovery testing.
Performance degradation on large CSV imports	Slow response or timeout.	Validate file size, process rows in batches, and return row-level import results.

3.8 Traceability to Requirements

Requirement Group	Primary Design Component	Design Reference
DFIP-F-001 to F-003; F-021	Authentication & RBAC	3.4, 4.3
DFIP-F-004 to F-006	Product / Inventory Service	3.4, 4.3

Requirement Group	Primary Design Component	Design Reference
DFIP-F-007 to F-009	Sales Data Service	3.4, 4.3
DFIP-F-010 to F-012	Demand Forecasting Engine	3.4, 4.2, 4.3
DFIP-F-013 to F-016	Inventory Planning Service	3.4, 4.2, 4.3
DFIP-F-017 to F-020	Dashboard / Report Service	3.4, 4.5
DFIP-F-022	Audit & Logging Service	3.4, 4.4
DFIP-SR-001 to SR-005	Authentication & RBAC; Data Validation / Preprocessing; Deployment / Transport Security	3.4, 3.9, 4.4
DFIP-F-023	Administration / Settings Service	3.4, 4.3

3.9 Security Architecture

Security is implemented as a cross-cutting concern across authentication, authorization, transport, validation, persistence and auditing. The architecture supports the SRS security objectives of protecting system data and limiting functions to authenticated and authorized users.

- Spoofing: require authenticated sessions and securely hashed passwords.
- Tampering / input validation (DFIP-SR-004): validate server-side input and imported CSV data, restrict write operations by role, and use database transactions.
- Repudiation: record important login, import, product, inventory and configuration changes in audit logs.
- Information disclosure / transport (DFIP-SR-005): avoid logging passwords or sensitive session data and use HTTPS/TLS when network deployed.
- Denial of service: apply file-size/request validation and reject malformed or excessive input.
- Elevation of privilege: enforce role checks on the backend for every restricted endpoint, not only in the UI.

4. Design

4.1 Design Overview

The runtime design follows a request-response flow. The web UI submits authenticated requests to backend APIs. The backend validates input and invokes the relevant application service. Data services read/write the relational database. Forecast requests additionally pass validated historical sales to the analytics engine. Planning requests combine the latest forecast with inventory and configured thresholds to produce reorder quantities and risk classifications. Forecast preprocessing aggregates demand to weekly or monthly periods and requires at least 8 periods. The forecasting engine uses Holt-Winters exponential smoothing when at least two seasonal cycles are available, otherwise simple exponential smoothing. Planning uses reorder_qty = max(0, ceil(forecast_demand + safety_stock - current_stock)). Risk is stockout when current_stock < forecast_demand, overstock when current_stock > overstock_factor x forecast_demand, otherwise none.

4.2 UML Sequence Diagrams

The following two sequence diagrams cover core project flows required by the template: demand forecast generation and inventory reorder/risk planning.

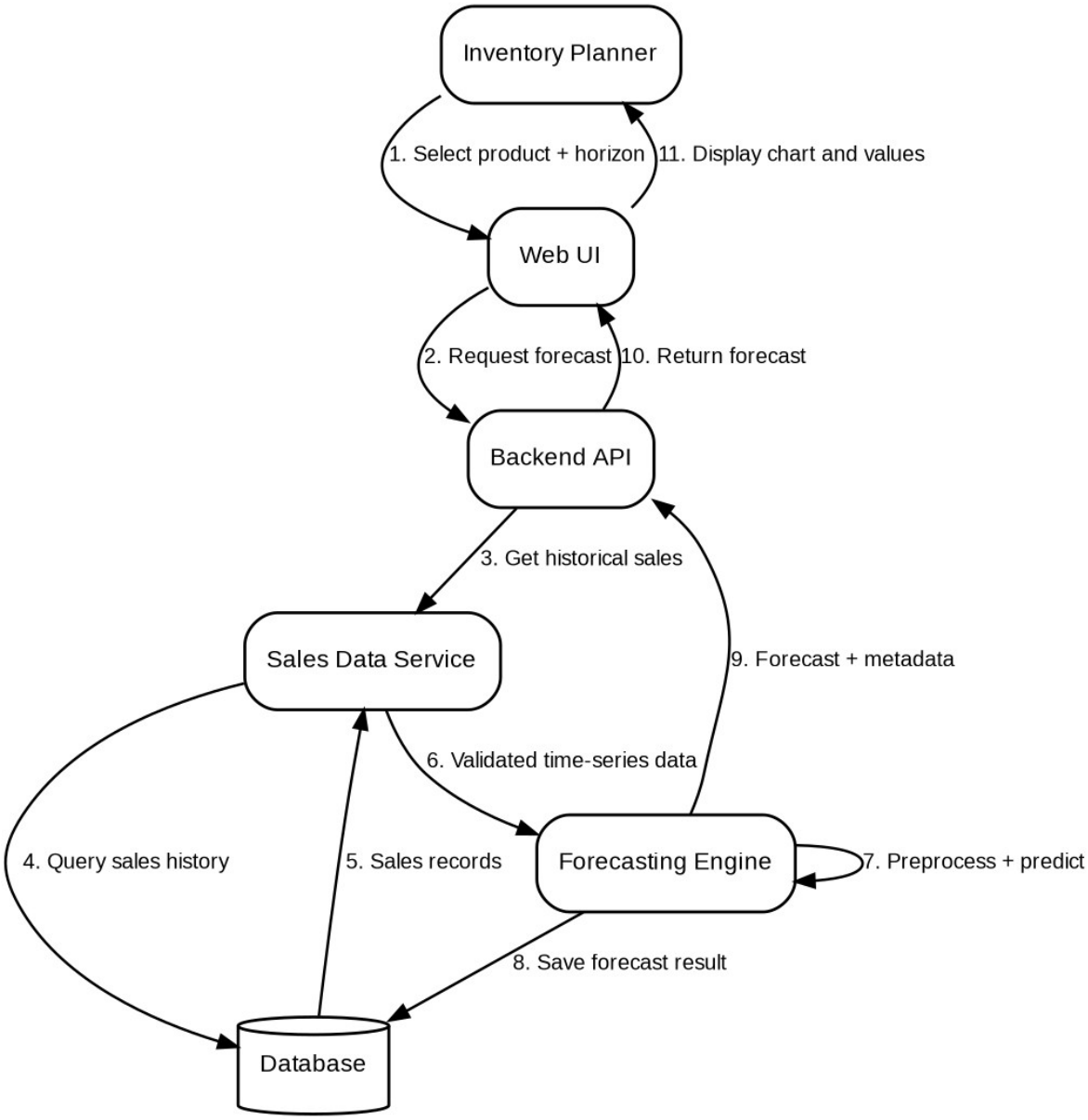


Figure 2. Sequence flow for generating a demand forecast.

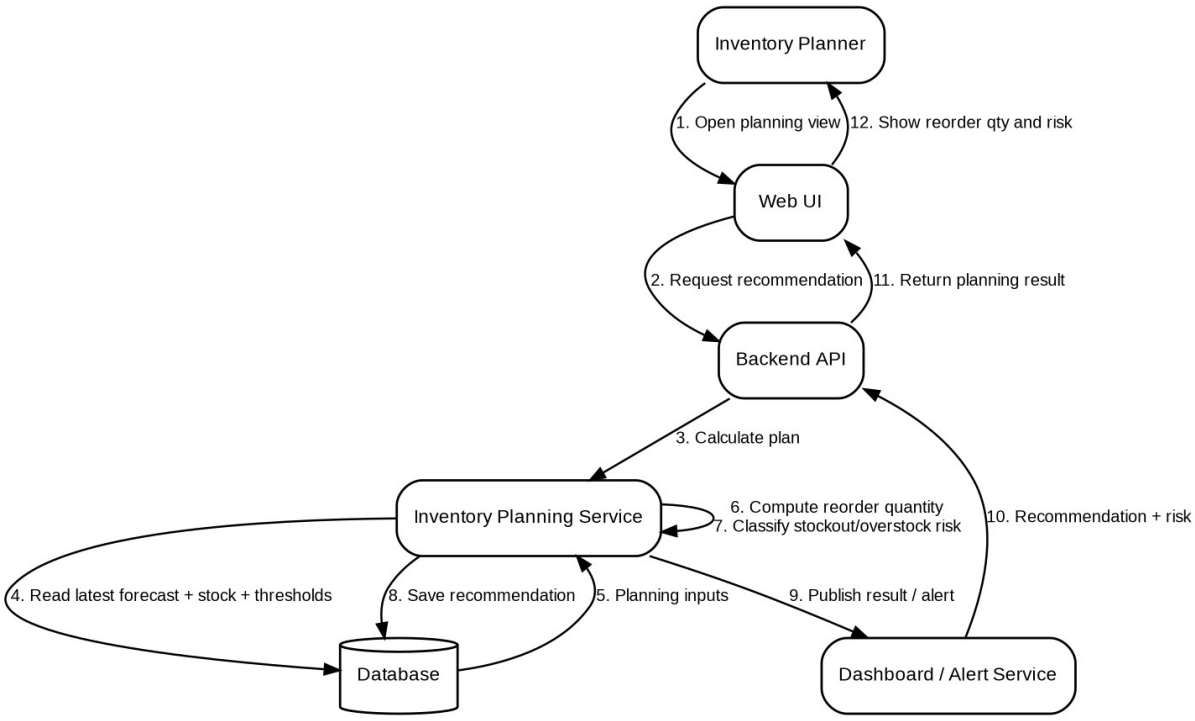


Figure 3. Sequence flow for reorder recommendation and risk classification.

4.3 API Design

The backend exposes REST-style interfaces. The exact URL prefix may be adjusted during implementation, but the following interfaces define the expected component contracts.

Endpoint	Method	Purpose	Example Request	Example Response / Errors
/api/auth/login	POST	Authenticate a user.	{"email":"user@example.com","password":"***"}	200 token; 401 invalid credentials.
/api/auth/logout	POST	Log out current user.	Bearer token	200 logout confirmation.
/api/users/	GET	List users (admin only).	Bearer admin token	200 user list; 403 non-admin.
/api/users/	POST	Create user with hashed password (admin only).	{"name":"Planner","email":"p @example.com","password": "***","role":"planner"}	201 user; 409 duplicate email; 403 non-admin.
/api/users/{user_id}/role	PUT	Change a user role (admin only).	{"role":"viewer"}	200 updated user; self-demotion blocked.
/api/users/{user_id}/ deactivate	PATCH	Deactivate a user (admin only).	Bearer admin token	200 deactivated user; self-deactivation blocked.
/api/products	GET	List active products.	Optional filters	200 product list.
/api/products	POST	Create product/SKU (admin only).	Product JSON	201 product; 422 invalid fields; 409 duplicate SKU.
/api/products/{id}	GET	Get one product.	Path product id	200 product; 404 missing.
/api/products/{id}	PUT	Update product (admin only).	Partial product JSON	200 product; 422 invalid fields; 404 missing.
/api/products/{id}/deactivate	PATCH	Deactivate product (admin only).	Path product id	200 result; 404 missing.

Endpoint	Method	Purpose	Example Request	Example Response / Errors
/api/inventory/{product_id}	GET	Read current inventory.	Path product id	200 inventory; 404 missing.
/api/inventory/{product_id}	PUT	Update inventory (admin/planner).	{"quantity":120}	200 inventory; 422 invalid quantity; 403 viewer.
/api/sales	POST	Add historical sales record.	{"product_id":12,"sale_date":"2026-09-01","quantity_sold":25}	201 sale; 422 invalid; 409 duplicate date/product.
/api/sales?product_id=&from=&to=	GET	List/filter historical sales.	Query parameters	200 filtered sales; 400 invalid range.
/api/sales/import	POST	Import CSV columns sku,date,quantity_sold (YYYY-MM-DD).	multipart/form-data CSV	200 import summary; 400 invalid CSV; 413 >2 MB.
/api/forecasts	POST	Generate demand forecast.	{"product_id":12,"horizon":4,"unit":"week"}	201 forecast; 400 invalid input; 422 insufficient history.
/api/forecasts?product_id=	GET	List stored forecasts for a product.	Query product_id	200 forecast list.
/api/planning/recommendations	POST	Calculate reorder quantity and risk.	{"product_id":12,"forecast_id":55}	200 reorder/risk; 404 missing data.
/api/planning/recommendations/{id}	GET	Get recommendation with all planning inputs.	Path recommendation id	200 recommendation; 404 missing.
/api/planning/alerts	GET	List current stockout/overstock risks.	Bearer token	200 alerts list.
/api/settings	GET	Read planning parameters (admin only).	Bearer admin token	200 settings; 403 non-admin.
/api/settings	PUT	Update planning parameters (admin only).	{"default_safety_stock":20,"overstock_factor":1.5,"min_history":8}	200 settings; 422 invalid; 403 non-admin.
/api/audit	GET	Query audit log (admin only).	actor/action/date filters	200 audit entries; 403 non-admin.

4.4 Error Handling, Logging & Monitoring

- Use consistent JSON error responses containing an error code, readable message and optional validation details.
- Reject invalid product IDs, negative quantities, malformed dates, unsupported horizons and invalid CSV schemas before business processing.
- Do not include passwords, password hashes or session secrets in application logs.
- Log authentication failures, data imports, product/inventory changes, forecast generation failures, planning errors and configuration changes with timestamps.
- Monitor request failures, forecast execution duration, CSV import errors and database connectivity during testing/deployment.
- Use database transactions for multi-step write operations where partial updates could create inconsistent records.

4.5 UX Design

- Login screen with clear validation messages and no exposure of password text.
- Dashboard cards for current inventory, forecast demand, reorder recommendations, stockout count and overstock count.
- Product and sales forms with required-field validation and understandable error messages.

- Forecast page with product selector, forecast horizon, predicted values, generation timestamp and a trend chart.
- Planning page that clearly separates recommended reorder quantity from stockout/overstock risk status.
- Reports page with filters for product and time period plus CSV export.
- Consistent navigation and readable labels suitable for first-time users.

4.6 Implementation Status & Next Steps

- Forecasting algorithm selected: Holt-Winters exponential smoothing for >= 2 seasonal cycles; simple exponential smoothing otherwise; minimum 8 periods; supported horizons 1-12 weeks or 1-6 months.
- Database schema is defined for users, products, inventory, sales, forecasts, forecast points, recommendations, audit logs and settings; integration testing will verify persistence and constraints.
- Core authentication, user administration, product, inventory and sales APIs are implemented; forecast/planning, audit/settings and frontend integration follow in the next PRs.
- Maintain SRS-SAD-STP traceability as remaining endpoints and tests are implemented.
- Automated tests cover forecasting, preprocessing, product, inventory, sales, authentication and user administration; planning and integration tests are added with their implementation PRs.
- Possible future enhancement: supplier lead-time modeling, automatic purchase-order generation and multi-location inventory support.

5. Appendices

5.1 Glossary

DFIP - Demand Forecasting & Inventory Planning; SKU - Stock Keeping Unit; RBAC - Role-Based Access Control; API - Application Programming Interface; TLS - Transport Layer Security; RTM - Requirements Traceability Matrix.

5.2 References

- Team 03 Software Requirements Specification (SRS), Version 1.1.
- Course-provided Software Architecture and Design Specification template.
- OWASP secure web application guidance (used as general security reference).

5.3 Tools

- Git/GitHub for version control and shared repository.
- draw.io or PlantUML/Graphviz for UML and architecture diagrams.
- Swagger/OpenAPI for API documentation.
- MySQL Workbench or pgAdmin for database management.
- Postman for API testing.